

Exp No.: 8 Fabric Network Deployment and Management

Aim

To deploy, manage, monitor, and maintain a Hyperledger Fabric network by creating the network, verifying network components, deploying chaincode, monitoring Docker containers, and performing network shutdown and cleanup operations.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

Hyperledger Fabric network deployment involves setting up the blockchain infrastructure required for enterprise applications. A Fabric network consists of peer nodes, ordering service, Certificate Authorities (CAs), channels, chaincode, and client applications. Network management includes:

- Starting and stopping the network
- Creating application channels
- Deploying chaincode
- Monitoring Docker containers
- Managing peer and orderer services
- Viewing network logs
- Cleaning up Docker containers and generated artifacts

Proper deployment and management ensure reliable blockchain operations and simplify maintenance and troubleshooting.

Procedure

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Verify Running Docker Containers

Display all active Docker containers.

```
docker ps
```

```

{
  "Admins": {
    "value": {
      "anchor_peers": [
        {
          "host": "peer0.org2.example.com",
          "port": 9051
        }
      ],
      "version": "0"
    },
    "MSP": {
      "mod_policy": "",
      "value": null,
      "version": "0"
    },
    "version": "1"
  },
  "mod_policy": "",
  "policies": {
    "values": {
      "version": "0"
    },
    "mod_policy": "",
    "policies": {
      "values": {
        "version": "0"
      }
    }
  }
}
++ configtxlator proto_encode --input /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Envelope --output /home/kali/fabric-dev/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-09-08 13:41:49.979 EDT 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
2026-09-08 13:41:49.996 EDT 0002 INFO [channelCmd] update → Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined

(kali@kali) ~/fabric-dev/fabric-samples/test-network
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
AMES
e2339e9d9b7b   hyperledger/fabric-peer:latest     "peer node start"       21 seconds ago Up 21 seconds 0.0.0.0:7051→7051/tcp, [::]:7051→7051/tcp, 0.0.0.0:9444→9444/tcp, [::]:9444→9444/tcp
6480d3b02c92   hyperledger/fabric-peer:latest     "peer node start"       21 seconds ago Up 21 seconds 0.0.0.0:9051→9051/tcp, [::]:9051→9051/tcp, 7051/tcp, 0.0.0.0:9445→9445/tcp, [::]:9445→9445/tcp
38a6702c6d76   hyperledger/fabric-orderer:latest  "orderer"               21 seconds ago Up 21 seconds 0.0.0.0:7050→7050/tcp, [::]:7050→7050/tcp, 0.0.0.0:7053→7053/tcp, [::]:7053→7053/tcp, 0.0.0.0:9443→9443/tcp, [::]:9443→9443/tcp
87954491a437   hyperledger/fabric-ca:latest       "sh -c 'fabric-ca-se..." 45 seconds ago Up 44 seconds 0.0.0.0:7054→7054/tcp, [::]:7054→7054/tcp, 0.0.0.0:17054→17054/tcp, [::]:17054→17054/tcp
08b9c3beb9b9   hyperledger/fabric-ca:latest       "sh -c 'fabric-ca-se..." 45 seconds ago Up 44 seconds 0.0.0.0:8054→8054/tcp, [::]:8054→8054/tcp, 7054/tcp, 0.0.0.0:18054→18054/tcp, [::]:18054→18054/tcp
56b0fb474bfa   hyperledger/fabric-ca:latest       "sh -c 'fabric-ca-se..." 45 seconds ago Up 44 seconds 0.0.0.0:9054→9054/tcp, [::]:9054→9054/tcp, 7054/tcp, 0.0.0.0:19054→19054/tcp, [::]:19054→19054/tcp
a_orderer

```

Figure 1: Running containers — peer0.org1, peer0.org2, orderer, and CA containers

Step 4: Display Docker Images

```
docker images
```

```
(kali㉿kali)-[~/fabric-dev/fabric-samples/test-network]
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
<none>	<none>	bb943d27dcf0	15 minutes ago	424MB
<none>	<none>	8d6ca36283d6	15 minutes ago	424MB
<none>	<none>	5933d3c3ef72	2 hours ago	424MB
<none>	<none>	62c767c819f7	2 hours ago	424MB
<none>	<none>	2d45f4c350e8	2 hours ago	424MB
<none>	<none>	b3dd64a00992	2 hours ago	424MB
<none>	<none>	356a58909a42	2 hours ago	424MB
<none>	<none>	06a0bfb2a47c	2 hours ago	424MB
<none>	<none>	a3d61a5b8870	2 hours ago	424MB
<none>	<none>	3ff5b48da434	2 hours ago	424MB
<none>	<none>	2aac8e9ee1fe	4 hours ago	424MB
<none>	<none>	a00b82a0ea48	4 hours ago	424MB
<none>	<none>	b29301514a23	2 weeks ago	424MB
<none>	<none>	a328966a24e5	2 weeks ago	424MB
hyperledger/fabric-peer	2.5	e382f4af4538	2 months ago	155MB
hyperledger/fabric-peer	2.5.16	e382f4af4538	2 months ago	155MB
hyperledger/fabric-peer	latest	e382f4af4538	2 months ago	155MB
hyperledger/fabric-orderer	2.5	80322dee2204	2 months ago	121MB
hyperledger/fabric-orderer	2.5.16	80322dee2204	2 months ago	121MB
hyperledger/fabric-orderer	latest	80322dee2204	2 months ago	121MB
hyperledger/fabric-ccenv	2.5	85e2ca8475b0	2 months ago	675MB
hyperledger/fabric-ccenv	2.5.16	85e2ca8475b0	2 months ago	675MB
hyperledger/fabric-ccenv	latest	85e2ca8475b0	2 months ago	675MB
hyperledger/fabric-baseos	2.5	71fca91e6713	2 months ago	160MB
hyperledger/fabric-baseos	2.5.16	71fca91e6713	2 months ago	160MB
hyperledger/fabric-baseos	latest	71fca91e6713	2 months ago	160MB
busybox	latest	c6348fa86ba0	3 months ago	4.45MB
hello-world	latest	e2ac70e7319a	5 months ago	10.1kB
hyperledger/fabric-ca	1.5	2161c222bb9e	7 months ago	244MB
hyperledger/fabric-ca	1.5.17	2161c222bb9e	7 months ago	244MB
hyperledger/fabric-ca	latest	2161c222bb9e	7 months ago	244MB
hyperledger/fabric-nodeenv	2.5	9450edc1d320	21 months ago	405MB

Figure 2: Hyperledger Fabric Docker images — peer, orderer, ccenv, baseos, and CA

Step 5: Deploy Asset Transfer Chaincode

```
./network.sh deployCC -ccl javascript -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript
```

Step 6: Verify Chaincode Approval and Commitment

Check the chaincode approval status across both organizations and verify commitment.

```
peer lifecycle chaincode checkcommitreadiness --channelID mychannel \
  --name basic --version 1.0 --sequence 1 --output json
```

```
kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help

    }
    "Org2MSP": false
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to check the commit readiness of the chaincode definition on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": false
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 2
+ peer lifecycle chaincode approveformyorg -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/kali/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --name basic --version 1.0 --package-id basic_1.0:1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629 --sequence 1
+ res=0
2026-09-08 13:30:31.964 EDT 0001 INFO [chaincodeCmd] ClientWait → txid [3695ba7a0f1736f35115c3877e8a71b5bbddc5361e541ecd1ef8af4010c3ef3] committed with status (VALID) at localhost:9051
Chaincode definition approved on peer0.org2 on channel 'mychannel'
Using organization 1
Checking the commit readiness of the chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to check the commit readiness of the chaincode definition on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true
  }
}
Checking the commit readiness of the chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Checking the commit readiness of the chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to check the commit readiness of the chaincode definition on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode checkcommitreadiness --channelID mychannel --name basic --version 1.0 --sequence 1 --output json
+ res=0
{
  "approvals": {
    "Org1MSP": true,
    "Org2MSP": true
  }
}
```

Figure 3: Chaincode approval by both organizations and commit readiness verification

Step 7: Verify Installed Chaincode

```
peer lifecycle chaincode queryinstalled
```

```
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
└─$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629, Label: basic_1.0
```

Figure 4: Installed chaincode — basic_1.0 with package ID

Step 8: Monitor Peer Logs

Display logs of the peer node.

```
docker logs peer0.org1.example.com
```

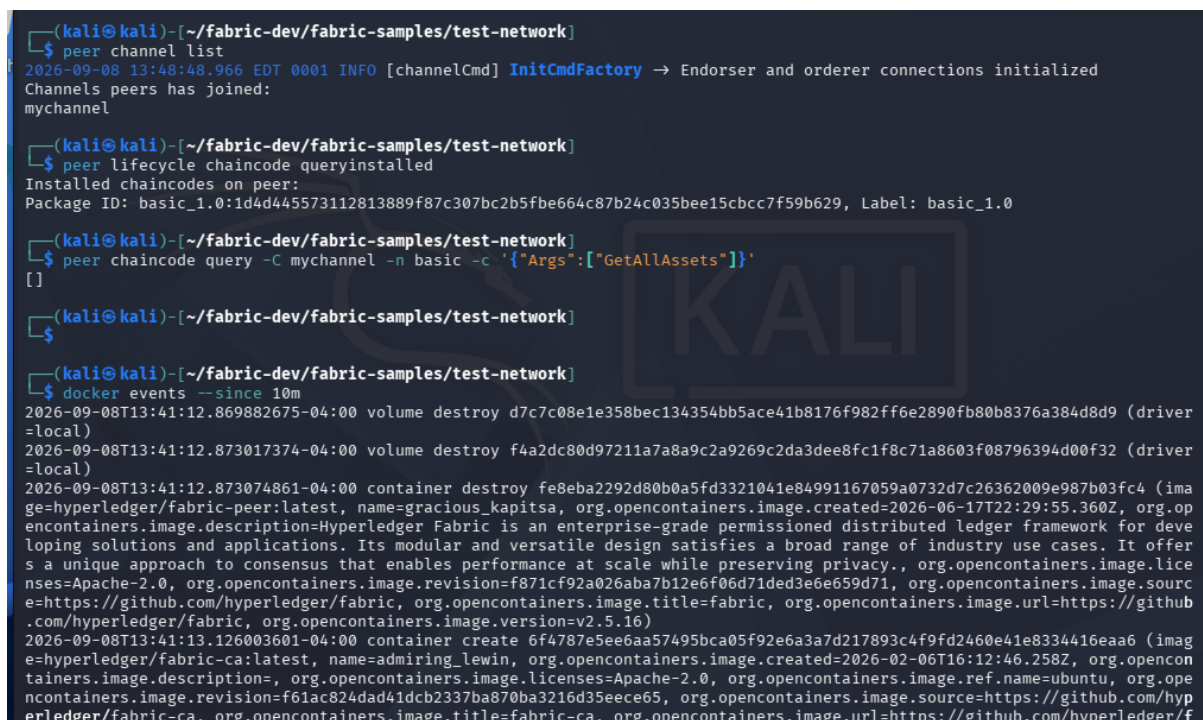
Step 9: Monitor Orderer Logs

```
docker logs orderer.example.com
```

Step 10: Verify the Channel and Monitor Docker Events

List joined channels, query the ledger, and observe real-time Docker events.

```
peer channel list
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
docker events --since 10m
```

A terminal window with a dark background and a 'KALI' watermark. It shows the execution of several commands in a Fabric test network. The first command, 'peer channel list', returns 'mychannel'. The second, 'peer lifecycle chaincode queryinstalled', shows installed chaincodes for 'basic_1.0'. The third, 'peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}', returns an empty array '[]'. The fourth, 'docker events --since 10m', shows a series of Docker events including volume destruction and container creation/destroy for Hyperledger Fabric components.

```
(kali@kali)~/fabric-dev/fabric-samples/test-network]
$ peer channel list
2026-09-08 13:48:48.966 EDT 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
Channels peers has joined:
mychannel

(kali@kali)~/fabric-dev/fabric-samples/test-network]
$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629, Label: basic_1.0

(kali@kali)~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[]

(kali@kali)~/fabric-dev/fabric-samples/test-network]
$
(kali@kali)~/fabric-dev/fabric-samples/test-network]
$ docker events --since 10m
2026-09-08T13:41:12.869882675-04:00 volume destroy d7c7c08e1e358bec134354bb5ace41b8176f982ff6e2890fb80b8376a384d8d9 (driver
=local)
2026-09-08T13:41:12.873017374-04:00 volume destroy f4a2dc80d97211a7a8a9c2a9269c2da3dee8fc1f8c71a8603f08796394d00f32 (driver
=local)
2026-09-08T13:41:12.873074861-04:00 container destroy fe8eba2292d80b0a5fd3321041e84991167059a0732d7c26362009e987b03fc4 (ima
ge=hyperledger/fabric-peer:latest, name=gracious_kapitsa, org.opencontainers.image.created=2026-06-17T22:29:55.360Z, org.op
encontainers.image.description=Hyperledger Fabric is an enterprise-grade permissioned distributed ledger framework for deve
loping solutions and applications. Its modular and versatile design satisfies a broad range of industry use cases. It offer
s a unique approach to consensus that enables performance at scale while preserving privacy., org.opencontainers.image.lice
nses=Apache-2.0, org.opencontainers.image.revision=f871cf92a026aba7b12e6f06d71ded3e6e659d71, org.opencontainers.image.sourc
e=https://github.com/hyperledger/fabric, org.opencontainers.image.title=fabric, org.opencontainers.image.url=https://github
.com/hyperledger/fabric, org.opencontainers.image.version=v2.5.16)
2026-09-08T13:41:13.126003601-04:00 container create 6f4787e5ee6aa57495bca05f92e6a3a7d217893c4f9fd2460e41e8334416eaa6 (ima
ge=hyperledger/fabric-ca:latest, name=admiring_lewin, org.opencontainers.image.created=2026-02-06T16:12:46.258Z, org.opencon
tainers.image.description=, org.opencontainers.image.licenses=Apache-2.0, org.opencontainers.image.ref.name=ubuntu, org.ope
ncontainers.image.revision=f61ac824dad41dcb2337ba870ba3216d35eece65, org.opencontainers.image.source=https://github.com/hyp
erledger/fabric-ca, org.opencontainers.image.title=fabric-ca, org.opencontainers.image.url=https://github.com/hyperledger/f
```

Figure 5: Channel membership, ledger query, and real-time Docker events

Step 11: Stop the Fabric Network

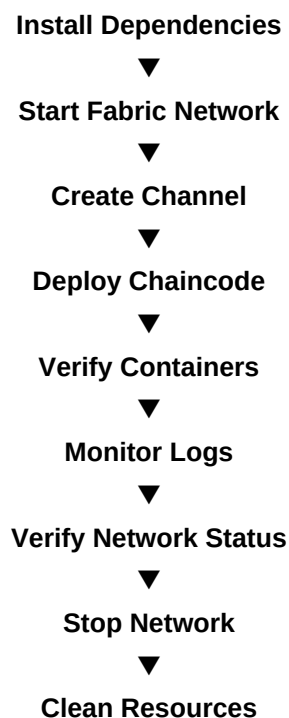
```
./network.sh down
```

Step 12: Clean Docker Resources (Optional)

Remove unused Docker containers, networks, and images.

```
docker system prune -f
```

Network Management Workflow



Result

The Hyperledger Fabric network was successfully deployed and managed. The blockchain network components, including peer nodes, ordering service, Certificate Authorities, and application channel, were verified. Chaincode was deployed successfully, network status was monitored, and the Fabric network was shut down and cleaned up successfully.